

Enterprise & Architectural Patterns

Repository · DTO · Service Layer · Unit of Work

Sub-Chapter 02·08

Repository	Hide database details behind a clean interface
DTO	Transfer only the data you need between layers
Service Layer	Put all business logic in one dedicated layer
Unit of Work	Group multiple DB changes into one transaction

1 Repository

Hide your data storage behind a clean interface

What it is

Repository hides how data is stored. The rest of the app only knows the interface -- not whether data comes from a database, file, or memory.

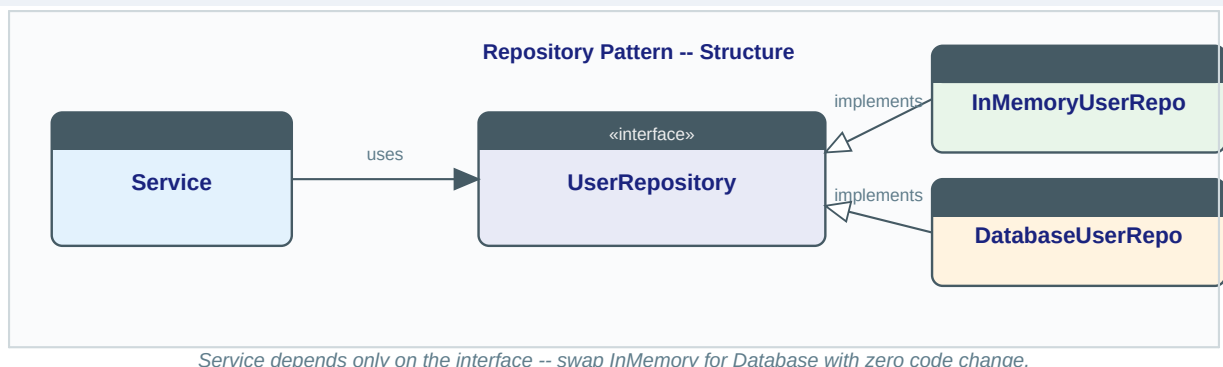
Simple Example (In-Memory Implementation)

```
// Domain object
class User {
    int id; String name;
    User(int id, String name) { this.id=id; this.name=name; }
    public String toString() { return "User{id="+id+", name="+name+"}"; }
}

// Repository interface -- the contract
interface UserRepository {
    void save(User user);
    User findById(int id);
    List<User> findAll();
}

// In-memory implementation (could swap to DB without touching calling code)
class InMemoryUserRepository implements UserRepository {
    private Map<Integer,User> store = new HashMap<>();
    public void save(User u) { store.put(u.id, u); }
    public User findById(int id) { return store.get(id); }
    public List<User> findAll() { return new ArrayList<>(store.values()); }
}

// Usage
UserRepository repo = new InMemoryUserRepository();
repo.save(new User(1, "Alice"));
repo.save(new User(2, "Bob"));
System.out.println(repo.findById(1)); // User{id=1, name=Alice}
System.out.println(repo.findAll()); // [User{id=1...}, User{id=2...}]
```



Key Rules

- Define a Repository interface; keep all DB/file code in the implementation class.
- The rest of the app imports only the interface -- never the concrete class.
- Swap implementations (in-memory for tests, SQL for production) without touching callers.

2 DTO -- Data Transfer Object

Send only the data the caller needs

What it is

Never expose your internal domain object across layers or APIs. Create a DTO that carries only what the caller needs.

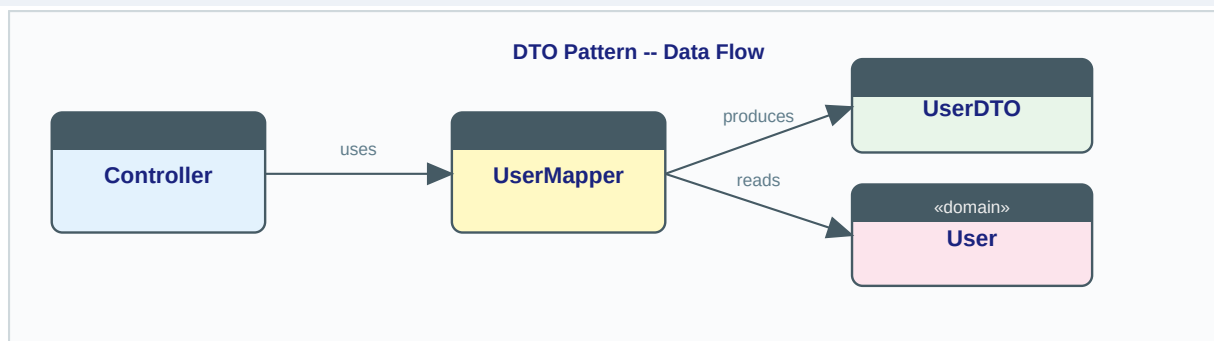
Simple Example

```
// Domain object (internal -- has sensitive fields)
class User {
    int id; String name; String password; String ssn;
    User(int id, String name, String password, String ssn) {
        this.id=id; this.name=name; this.password=password; this.ssn=ssn;
    }
}

// DTO -- only safe fields for the API response
class UserDTO {
    int id; String name;
    UserDTO(int id, String name) { this.id=id; this.name=name; }
    public String toString() { return "UserDTO{id="+id+", name="+name+"}"; }
}

// Mapper -- converts domain -> DTO
class UserMapper {
    static UserDTO toDTO(User user) {
        return new UserDTO(user.id, user.name);
        // password and ssn are NOT included
    }
}

// Usage
User user = new User(1, "Alice", "secret123", "123-45-6789");
UserDTO dto = UserMapper.toDTO(user);
System.out.println(dto); // UserDTO{id=1, name=Alice}
// Password and SSN never leave the server
```



Controller receives/returns UserDTO -- the domain User (with password/SSN) never leaves the server.

Key Rules

- Create a separate DTO class with only the fields the API caller should see.
- Use a Mapper class to convert between domain objects and DTOs.
- Never return domain objects directly from controllers or REST endpoints.

3 Service Layer

One dedicated layer owns all business logic

What it is

Service layer sits between the controller (web layer) and the repository (data layer). It owns all business logic.

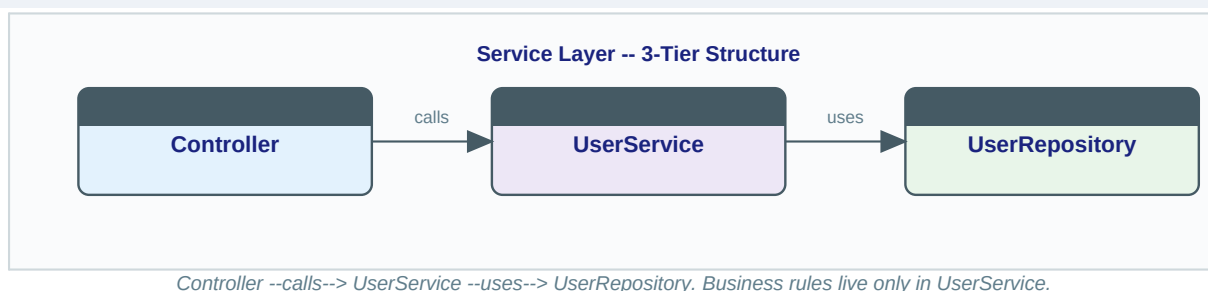
Simple Example

```
// Repository
interface UserRepository { User findById(int id); void save(User u); }

// Service -- all business logic lives here
class UserService {
    private UserRepository repo;
    UserService(UserRepository repo) { this.repo = repo; }

    UserDTO getUser(int id) {
        User user = repo.findById(id);
        if (user == null) throw new RuntimeException("User not found: " + id);
        return UserMapper.toDTO(user); // returns DTO, not domain object
    }
    void registerUser(String name) {
        if (name == null || name.isBlank())
            throw new IllegalArgumentException("Name cannot be empty");
        User user = new User(0, name, "", "");
        repo.save(user);
        System.out.println("Registered: " + name);
    }
}

// Controller only calls service -- never touches repo directly
class UserController {
    private UserService service;
    UserController(UserService service) { this.service = service; }
    void handleRegister(String name) { service.registerUser(name); }
}
```



Key Rules

- Controllers must not contain business rules -- delegate everything to the Service.
- Services must not build HTTP responses -- delegate that back to the Controller.
- Repositories must not contain business rules -- they only read/write data.

4 Unit of Work

Commit all DB changes together or roll back

What it is

Collect multiple changes (inserts, updates, deletes) and commit them all in one go. If anything fails, roll everything back.

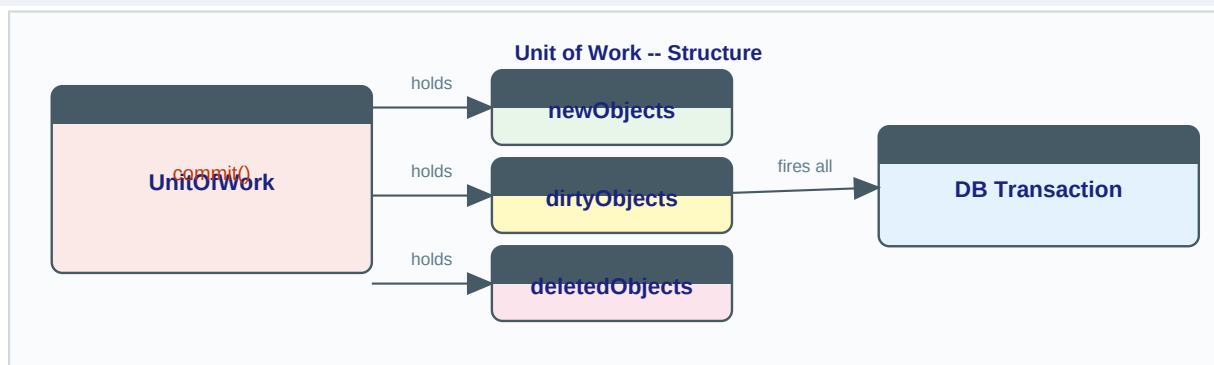
Simple Example

```
class UnitOfWork {
    private List<Runnable> newObjects      = new ArrayList<>();
    private List<Runnable> dirtyObjects    = new ArrayList<>();
    private List<Runnable> deletedObjects = new ArrayList<>();

    void registerNew(Runnable insertAction)    { newObjects.add(insertAction); }
    void registerDirty(Runnable updateAction)  { dirtyObjects.add(updateAction); }
    void registerDeleted(Runnable deleteAction) { deletedObjects.add(deleteAction); }

    void commit() {
        System.out.println("-- Starting transaction --");
        try {
            newObjects.forEach(Runnable::run);
            dirtyObjects.forEach(Runnable::run);
            deletedObjects.forEach(Runnable::run);
            System.out.println("-- Committed! --");
        } catch (Exception e) {
            System.out.println("-- Rolled back! --");
        }
    }
}

// Usage
UnitOfWork uow = new UnitOfWork();
uow.registerNew( () -> System.out.println("INSERT user Alice"));
uow.registerDirty( () -> System.out.println("UPDATE user Bob"));
uow.registerDeleted(() -> System.out.println("DELETE user Charlie"));
uow.commit();
// -- Starting transaction --
// INSERT user Alice / UPDATE user Bob / DELETE user Charlie
// -- Committed! --
```



commit() fires newObjects, dirtyObjects, deletedObjects in one transaction -- all succeed or all fail.

Key Rules

- Register every change (insert/update/delete) before calling commit().
- commit() wraps all actions in a single transaction -- partial updates are never saved.

- On failure inside `commit()`, catch the exception and roll back all changes.

* Cheat Sheet

Quick reference for all four Enterprise Patterns

Pattern	Intent	Problem it solves	Key class	Real Spring example
Repository	Hide storage details	Coupling to DB technology	UserRepository	@Repository / JpaRepository
DTO	Transfer minimal data	Leaking sensitive domain fields	UserDTO	@ResponseBody with DTO class
Service Layer	Centralise business logic	Logic scattered in controllers	UserService	@Service
Unit of Work	Atomic multi-step DB ops	Partial saves on failure	UnitOfWork	@Transactional